“TUGAS PRATIKUM 6 SISTEM OPERASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Operasi

Dosen : Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

NAMA : Reandra Khansa Faadihilah

NIM : 2410651060

KELAS : TI – C

Tanggal : 17-11-2025

UNIVERSITAS MUHAMMADIYAH JEMBER

2025-2026

DAFTAR ISI

Tugas 1 : Analisis Deadlock

Pada tugas 1 kali ini kita belajar bagaimana cara memahami bagaimana deadlock bekerja dalam system operasi melalui simulasi sederhana menggunakan pthread mutex deangan menjalankan deadlock_simple.c

- Mengapa program mengalami deadlock?

  Program mengalami deadlock karena kedua thread mengambil lock dalam urutan yang berbeda

  1. Thread 1: mengambil Lock 1 hingga menunggu lock 2
  2. Thread 2 : mengambil Lock 2 hingga menunggu Lock 1

  Keduanya saling menunggu sehingga tidak ada yang dapat melanjutkan. Karena kedua thread tidak pernah melepaskan lock yang sudah dimilikinya, program berhenti (hang)

- Kondisi deadlock apa saja yang terpenuhi?

  1. Mutual Exlusion
     Mutex hanya dapat dipegang satu thread pada satu waktu
  2. Hold and Wait
     Thread 1 memegang Lock1 sambil menunggu Lock2, dan sebaliknya
  3. No. Preemption
     Lock tidak dapat direbut paksa oleh thread lain
  4. Cicular Wait
     T1 menunggu L2, T2 menunggu L1 sehingga membentuk lingkaran

- Gambar diagram resource allocation graph-nyaI

Reprentasi graf	Visual lingkarannya
T1 → Lock1 T2 → Lock2 T1 ← Lock2 T2 ← Lock1	(T1) ----> [Lock1] ^ | | **v** [Lock2] <---- (T2)

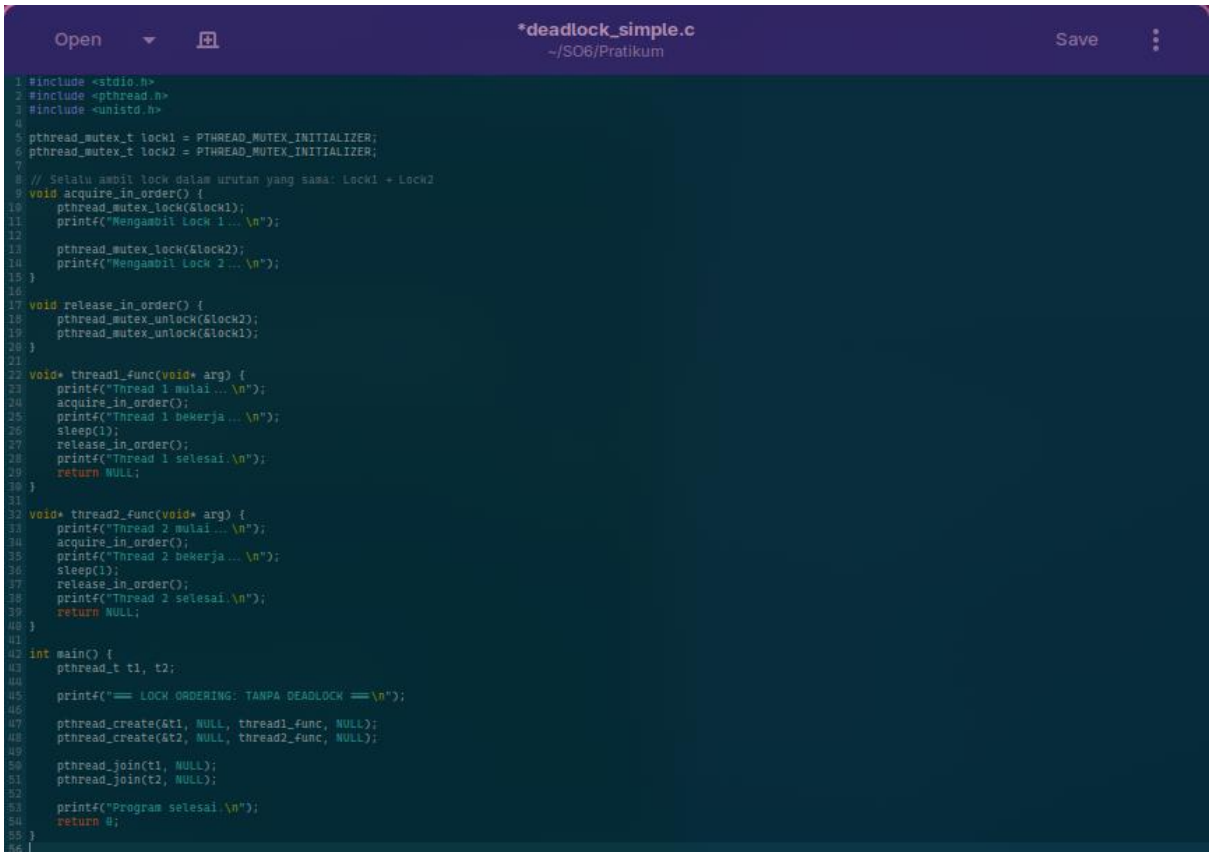
Tugas 2 : Implementasi Pencegahan

Pada tugas ke-2 ini kita berfokus pada pencegahan terjadinya deadlock. Setelah memahami bagaimana deadlock terjadi pada tugas ke-1 kita memodifkasi program agar tidak lagi mengalami deadlock dengan dua teknik Lock ordering dan Try Lock dengan Retry

PRAKTIK

Modifikasi Lock Ordering

Memodifkasi file deadlock_simple.c



COMPILE PROGRAM DAN OUTPUTNYA :

```
➜ ~/SO6/Pratikum ❯ gcc deadlock_simple.c -o deadlock_lckordering -lpthread
➜ ~/SO6/Pratikum ❯ ./deadlock_lckordering                          c -gcc ⊙ 19:39
═══ LOCK ORDERING: TANPA DEADLOCK ═══
Thread 1 mulai ...
Mengambil Lock 1 ...
Mengambil Lock 2 ...
Thread 1 bekerja ...
Thread 2 mulai ...
Thread 1 selesai.
Mengambil Lock 1 ...
Mengambil Lock 2 ...
Thread 2 bekerja ...
Thread 2 selesai.
Program selesai.
➜ ~/SO6/Pratikum ❯                                                 c -gcc ⊙ 19:39
```

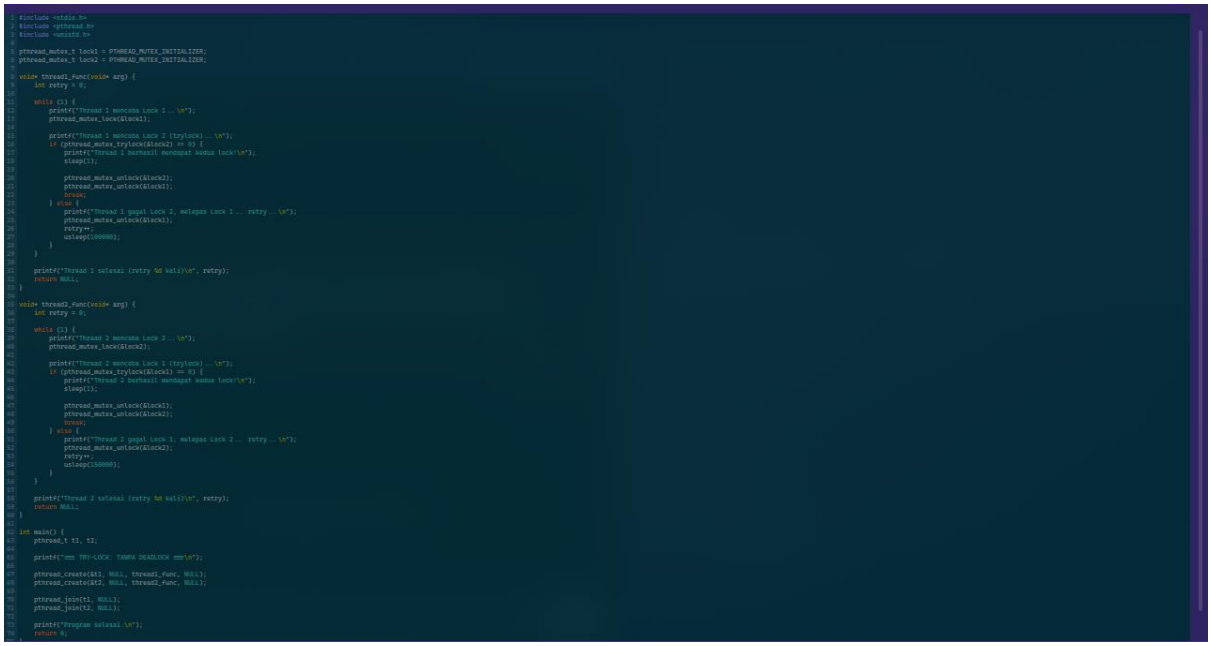
PENJELASAN

Program tersebut berkerja menghilangkan circular wait dengan memastikan semua thread mengambil locks dalam urutan yang sama sehingga deadlock tidak dapat terbentuk

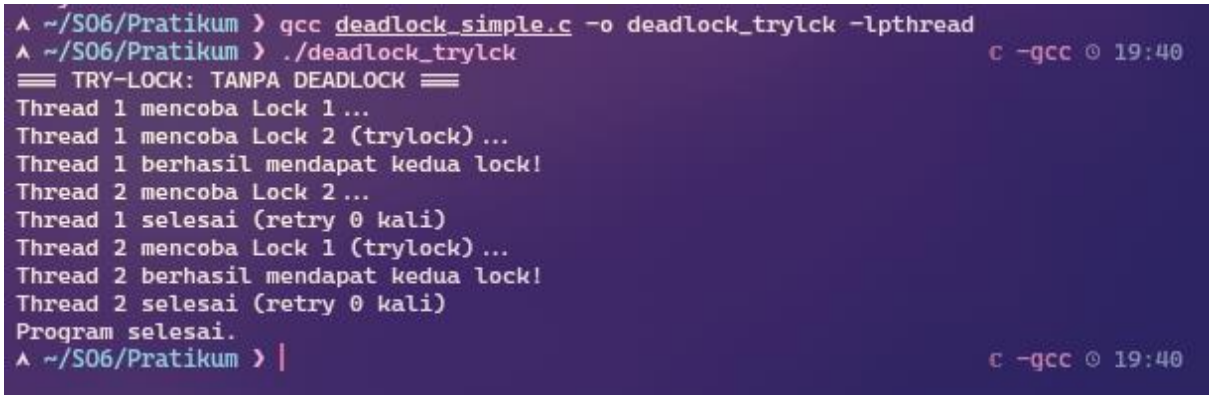
Membuat dua helper acquire_in_order() dan release_in_order() dengan memastikan pengambilan lock 1 dan lock 2

Modifikasi Try-Lock dengan retry

Memodifkasi deadlock_simple.c



COMPILE PROGRAM DAN OUTPUTNYA :



PENJELASAN

Thread melakukan pthread_mutex_lock pada lock pertama (blocking), lalu pthread_mutex_trylock pada lock kedua (non-blocking).

Jika trylock gagal, thread segera pthread_mutex_unlock lock pertama dan menunggu (usleep), lalu ulang loop.

Ketika trylock sukses, thread melakukan critical section, lalu melepaskan kedua lock dan keluar loop

Perbandingan Dari Ketiga Program

1. Program Deadlock Sederhana deadlock_simple.c
    a) Dua thread saling mengambil kunci (lock) dengan urutan berbeda.
    b) Akibatnya, mereka bisa saling menunggu dan program macet (deadlock).
    c) Tidak ada mekanisme pencegahan.
2. Program Lock Ordering deadlock_prevention_ordering.c
    a) Kedua thread mengambil kunci dengan urutan yang sama:
    b) Lock 1 dulu, baru Lock 2.
    c) Karena urutan sama, tidak ada yang saling menunggu.
    d) Program pasti aman dan tidak macet.
3. Program Try-Lock deadlock_prevention_trylock.c
    a) Thread mencoba mengambil lock kedua.
    b) Jika gagal, dia lepas lock pertama lalu coba lagi.
    c) Tidak akan deadlock, tetapi bisa perlu coba berkali-kali.

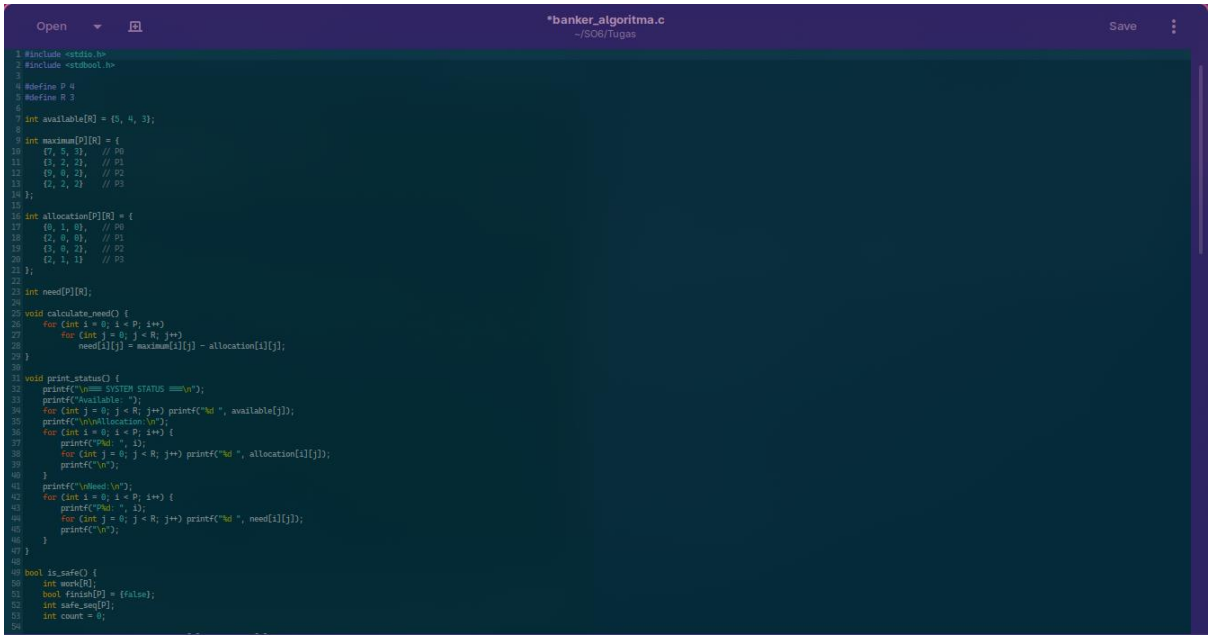
Tugas 3 : Banker’s Algorithm

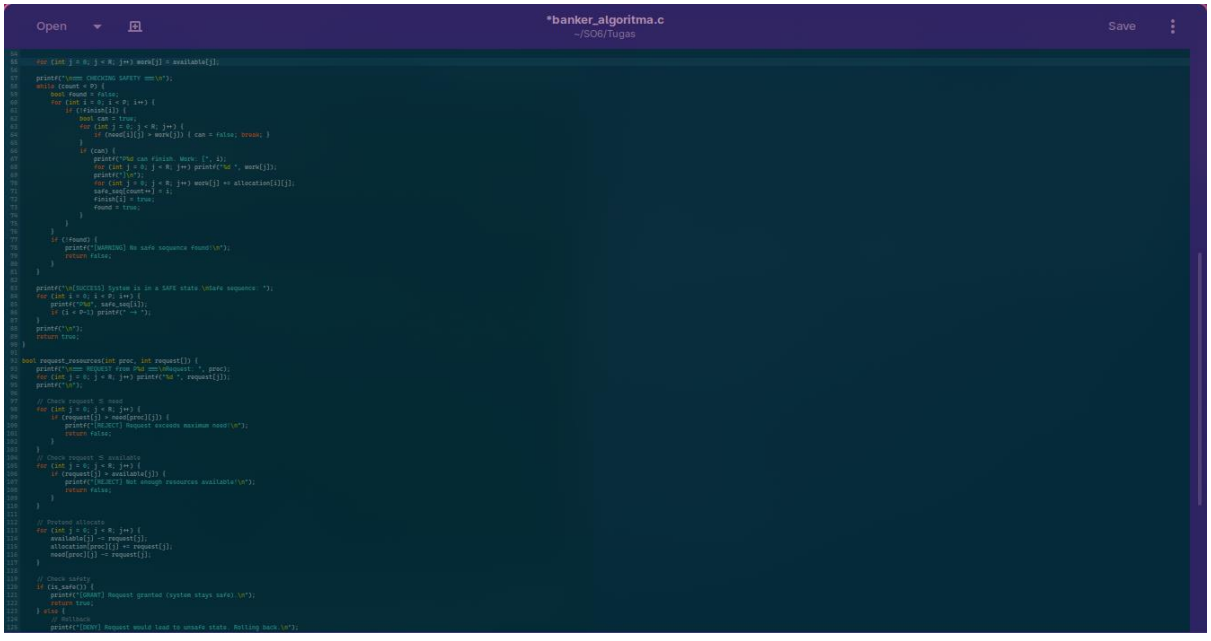
Pada tugas ke-3 kali ini kita membuat Banker Algorithm, Algoritma yang digunakan sistem operasi untuk menentukan apakah suatu permintaan resource aman atau tidak jika diberikan, dengan membuat program menghitung need, memeriksa safe state, dan menentukan apakah request dapat dikabulkan

PRAKTIK

Membuat file dengan nama banker_algoritma.c dengan text editor gedit

```
➜ ~/SO6/Tugas ❯ gedit banker_algoritma.c   ⊙ 19:48
```





```c
        // Rollback
        printf("[DENY] Request would lead to unsafe state. Rolling back.\n");
        for (int j = 0; j < R; j++) {
            available[j] += request[j];
            allocation[proc][j] -= request[j];
            need[proc][j] += request[j];
        }
        return false;
    }
}

int main() {
    printf("=== BANKER'S ALGORITHM - Tugas 3 ===\n");
    calculate_need();
    print_status();

    // Initial safety check
    is_safe();

    // Example request 1: safe request by P1
    int r1[R] = {1, 0, 2};
    request_resources(1, r1);
    print_status();

    // Example request 2: unsafe request by P2 (for demonstration)
    int r2[R] = {3, 3, 0};
    request_resources(2, r2);
    print_status();

    printf("\n=== PROGRAM FINISHED ===\n");
    return 0;
}
```

COMPILE FILE DAN OUTPUTNYA

```
~/SOS/Tugas ❯ gcc banker_algoritma.c -o banker_algoritma          c -gcc 0 19:53
~/SOS/Tugas ❯ ./banker_algoritma                                  c -gcc 0 19:53
=== BANKER'S ALGORITHM - Tugas 3 ===

=== SYSTEM STATUS ===
Available: 5 4 3

Allocation:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1

=== CHECKING SAFETY ===
P1 can finish. Work: [5 4 3 ]
P2 can finish. Work: [7 4 3 ]
P3 can finish. Work: [10 4 5 ]
P0 can finish. Work: [12 5 5 ]

[SUCCESS] System is in a SAFE state.
Safe sequence: P1 → P2 → P3 → P0

=== REQUEST from P1 ===
Request: 1 0 2

=== CHECKING SAFETY ===
P1 can finish. Work: [4 4 1 ]
P2 can finish. Work: [7 4 3 ]
P3 can finish. Work: [10 4 5 ]
P0 can finish. Work: [12 5 5 ]

[SUCCESS] System is in a SAFE state.
Safe sequence: P1 → P2 → P3 → P0
[GRANT] Request granted (system stays safe).

=== SYSTEM STATUS ===
Available: 4 4 1

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 0 2 0
P2: 6 0 0
P3: 0 1 1

=== REQUEST from P2 ===
Request: 3 3 0
[REJECT] Request exceeds maximum need!

=== SYSTEM STATUS ===
Available: 4 4 1

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 0 2 0
P2: 6 0 0
P3: 0 1 1

=== PROGRAM FINISHED ===
~/SOS/Tugas ❯
```

PENJELASAN :

Program menghitung need = maximum - allocation.

Menampilkan status, memeriksa apakah sistem aman.

Melakukan 2 request contoh:

Request 1 dari P1: {1,0,2} → Jika aman, akan diberikan.

Request 2 dari P2: {3,3,0} → kemungkinan ditolak (Tidak aman) dan akan di-rollback.

Output akan menunjukkan safe sequence jika aman, atau pesan peringatan jika tidak

Tugas 4 : Kasus Nyata (Real Case)

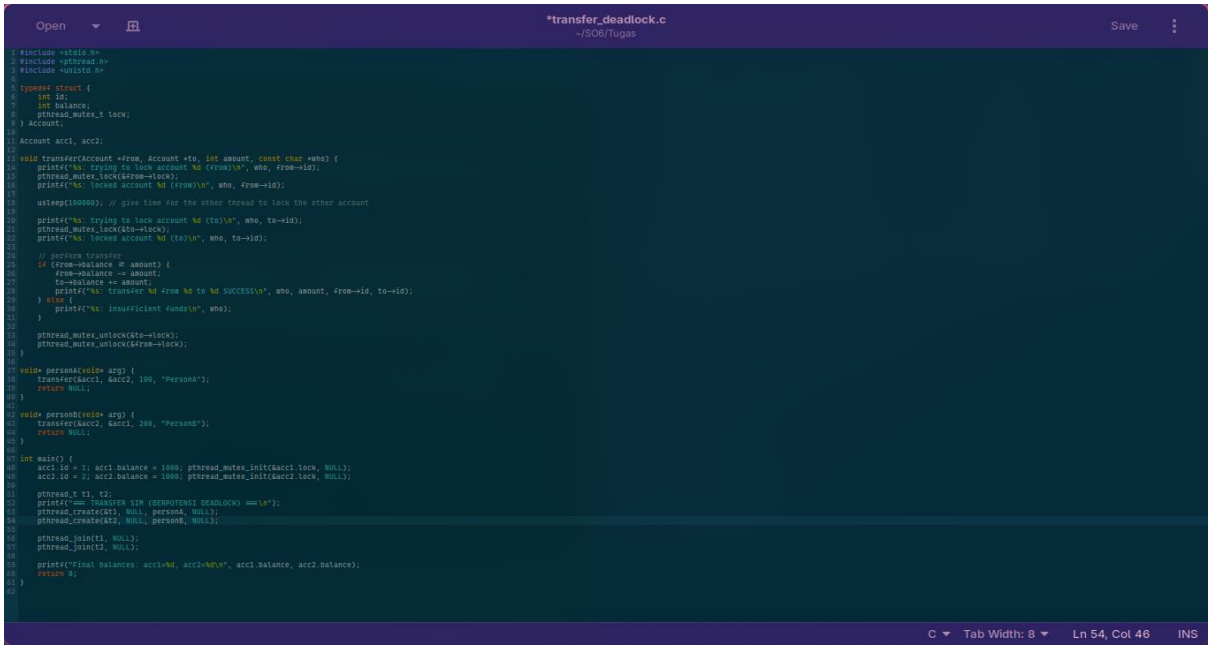
Pada tugas ke-4 kali ini kita mesimulasikan proses dimana dua orang melalukan transfer uang antar rekening secara bersamaan, Jika dua proses mengambil kunci rekening secara berbeda urutan maka deadlock dapat terjadi, pada tugas kali ini kita menerapkan teknik pencegahan agar kasus tersebut dapat berjalan tanpa hang.

PRAKTIK

Membuat file **transfer_deadlock.c** dengan text editor gedit

Program sengaja dibuat deadlock untuk pengujian

```
➜ ~/SO6/Tugas ❯ gedit transfer_deadlock.c
```




```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    int id;
    int balance;
    pthread_mutex_t lock;
} Account;

Account acc1, acc2;

void transfer(Account *from, Account *to, int amount, const char *who) {
    printf("%s: trying to lock account %d (from)\n", who, from->id);
    pthread_mutex_lock(&from->lock);
    printf("%s: locked account %d (from)\n", who, from->id);

    usleep(100000); // give time for the other thread to lock the other account

    printf("%s: trying to lock account %d (to)\n", who, to->id);
    pthread_mutex_lock(&to->lock);
    printf("%s: locked account %d (to)\n", who, to->id);

    // perform transfer
    if (from->balance >= amount) {
        from->balance -= amount;
        to->balance += amount;
        printf("%s: transfer %d from %d to %d SUCCESS\n", who, amount, from->id, to->id);
    } else {
        printf("%s: insufficient funds\n", who);
    }

    pthread_mutex_unlock(&to->lock);
    pthread_mutex_unlock(&from->lock);
}

void* personA(void* arg) {
    transfer(&acc1, &acc2, 100, "PersonA");
    return NULL;
}

void* personB(void* arg) {
    transfer(&acc2, &acc1, 200, "PersonB");
    return NULL;
}

int main() {
    acc1.id = 1; acc1.balance = 1000; pthread_mutex_init(&acc1.lock, NULL);
    acc2.id = 2; acc2.balance = 1000; pthread_mutex_init(&acc2.lock, NULL);

    pthread_t t1, t2;
    printf("=== TRANSFER SIM (BERPOTENSI DEADLOCK) ===\n");
    pthread_create(&t1, NULL, personA, NULL);
    pthread_create(&t2, NULL, personB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final balances: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);
    return 0;
}
```

COMPILE DAN OUTPUNYA :

```
➜ ~/SO6/Tugas ❯ gcc transfer_deadlock.c -o transfer_deadlock                c -gcc ⏲ 19:57
➜ ~/SO6/Tugas ❯ ./transfer_deadlock                                         c -gcc ⏲ 19:57
=== TRANSFER SIM (BERPOTENSI DEADLOCK) ===
PersonA: trying to lock account 1 (from)
PersonA: locked account 1 (from)
PersonB: trying to lock account 2 (from)
PersonB: locked account 2 (from)
PersonA: trying to lock account 2 (to)
PersonB: trying to lock account 1 (to)
^C
➜ ~/SO6/Tugas ❯                                                             c -gcc ⏲ 19:57
```

PENJELASAN :

Program ini sengaja dibuat untuk mengalami deadlock dan akan macet ketika dijalankan bersamaan oleh kedua thread.

Thread PersonA mengunci acc1 → sleep → coba kunci acc2

Thread PersonB mengunci acc2 → sleep → coba kunci acc1

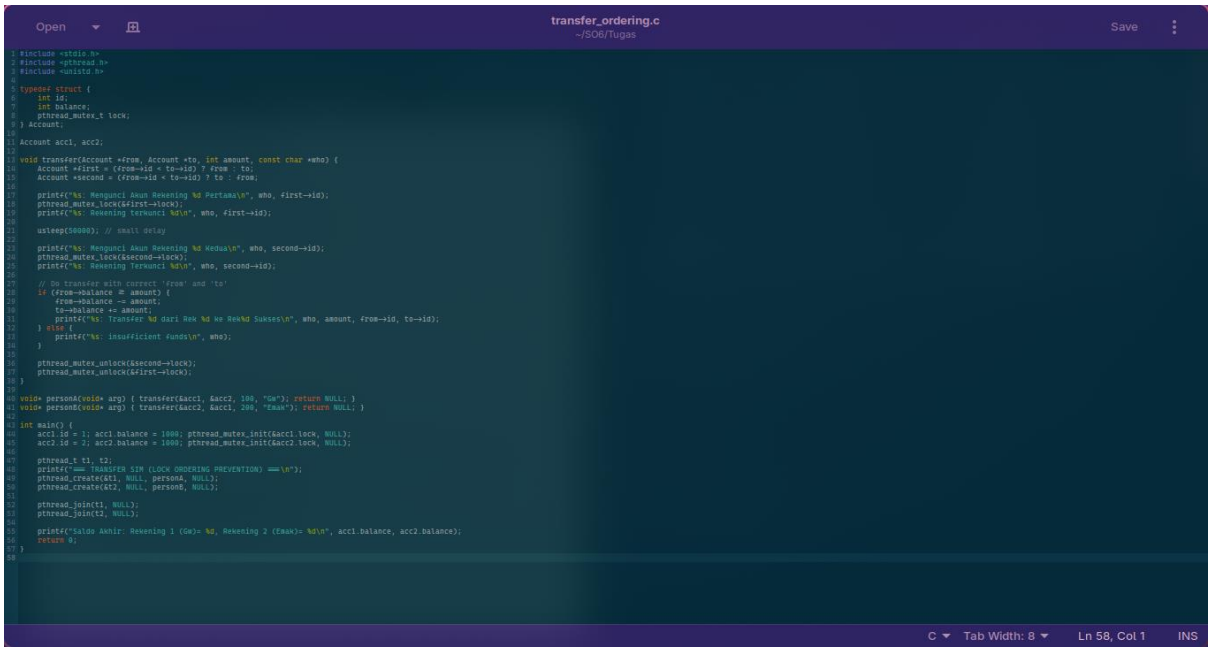
→ Terjadi circular wait → kedua thread saling menunggu selamanya.

Memodifikasi Program dengan mencgah deadlock sepenuhnya dengan lock ordering

PRAKTIK

Membuat program dengan nama **transfer_ordering.c**

```
A ~/SO6/Tugas ❯ gedit transfer_ordering.c
```



COMPILE DAN HASILNYA :

```
A ~/SO6/Tugas ❯ gcc transfer_ordering.c -o transfer_ordering
A ~/SO6/Tugas ❯ ./transfer_ordering
=== TRANSFER SIM (LOCK ORDERING PREVENTION) ===
Gw: Mengunci Akun Rekening 1 Pertama
Gw: Rekening terkunci 1
Emak: Mengunci Akun Rekening 1 Pertama
Gw: Mengunci Akun Rekening 2 Kedua
Gw: Rekening Terkunci 2
Gw: Transfer 100 dari Rek 1 ke Rek2 Sukses
Emak: Rekening terkunci 1
Emak: Mengunci Akun Rekening 2 Kedua
Emak: Rekening Terkunci 2
Emak: Transfer 200 dari Rek 2 ke Rek1 Sukses
Saldo Akhir: Rekening 1 (Gw)= 1100, Rekening 2 (Emak)= 900
A ~/SO6/Tugas ❯
```

PENJELASAN

Program ini mencegah deadlock dengan teknik lock ordering(mengunci akun dengan Idyang lebih dahulu)

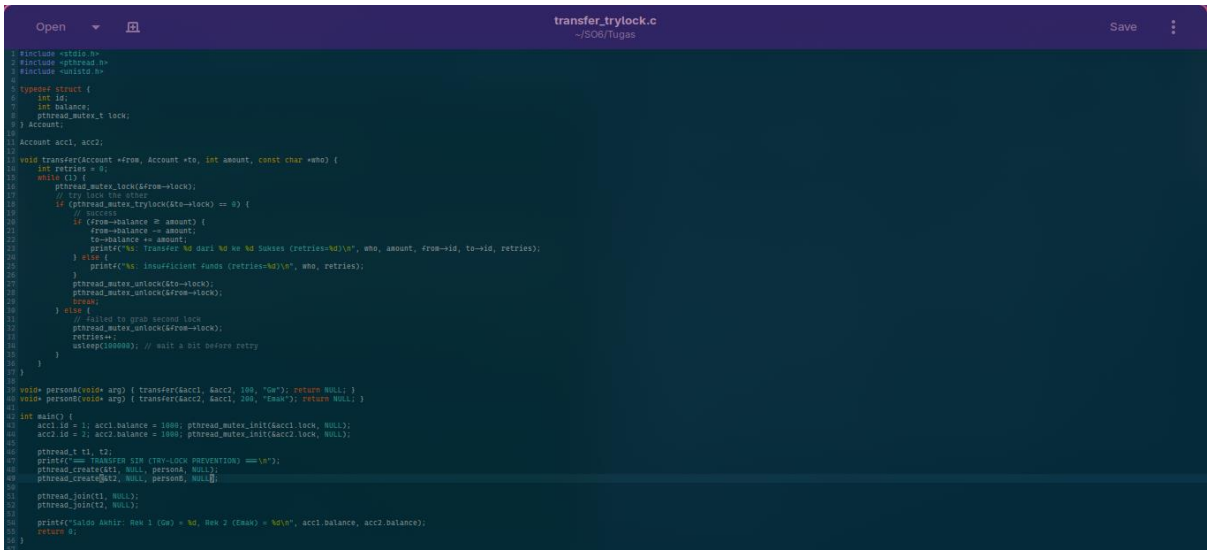
Kedua thread melakukan transfer silang acc1 ke acc2 dan acc2 ke acc1 tanpa adanya circular wait dengan mengurutkan penguncian selalu sama

Membuat program dengan mencegah deadlock dengan teknik trylock

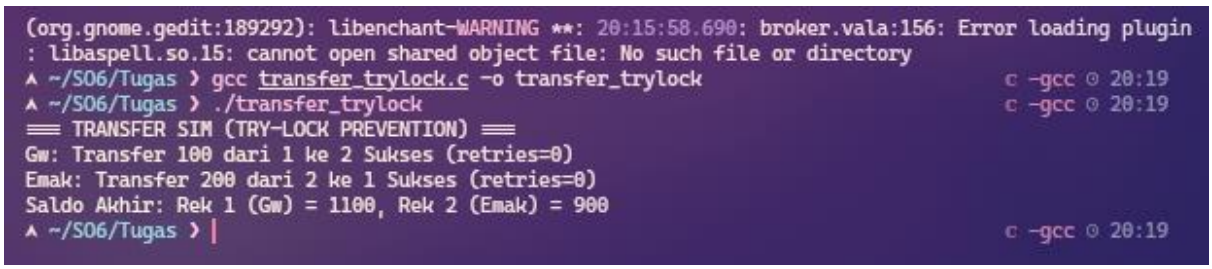
PRAKTIK

Membuat program dengan nama transfer_trylock.c

```
~/SO6/Tugas ❯ gedit transfer_trylock.c
```



HASIL & COMPILENYA :



PENJELASAN :

Pada program ke-3 ini mencegah adanya deadlock dengan teknik trylock + rollback + retry

Thread mengunci akun pertama dengan pthread_mutex_lock

Mencoba mengunci akun kedua dengan pthread_mutex_trylock

Jika gagal → lepas lock pertama → tunggu sebentar → coba lagi dari awal

Dengan adanya program transfer_trylock.c tidak pernah memegang satu lock sambil menunggu lock lain secara permanen